PRACTICAL - 05

Aim: Shape areas (Shape.java + Driver.java)

Code Screenshot:

Shape.java

```java
public abstract class Shape {
    public abstract double area();
    public abstract String getName();
}

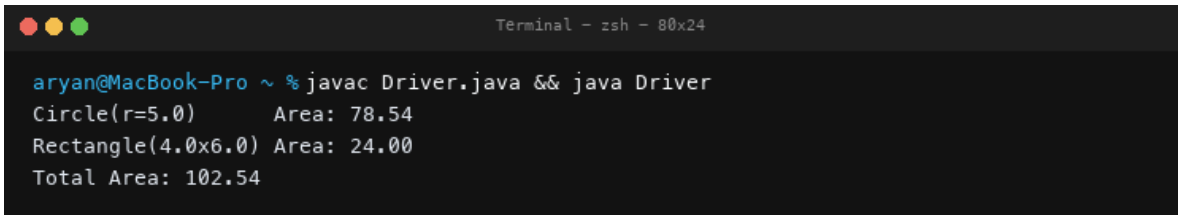
class Circle extends Shape {
    private double r;
    public Circle(double r) { this.r = r; }
    public double area() { return Math.PI * r * r; }
    public String getName() { return "Circle(r=" + r + ")"; }
}

class Rectangle extends Shape {
    private double w, h;
    public Rectangle(double w, double h) { this.w = w; this.h = h; }
    public double area() { return w * h; }
    public String getName() { return "Rectangle(" + w + "x" + h + ")"; }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(5.0), new Rectangle(4.0, 6.0) };
        double total = 0;
        for (Shape s : shapes) {
            total += s.area();
            System.out.printf("%-18s Area: %.2f\n", s.getName(), s.area());
        }
        System.out.printf("Total Area: %.2f\n", total);
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood abstract classes and abstract method declarations.
- Mastered runtime polymorphism and dynamic method dispatch.
- Learned to process heterogeneous collections using base-class references.
- Implemented aggregation and extremum tracking in polymorphic loops.
- Understood the Open-Closed Principle (OCP) using class hierarchies.

Aim: Payroll (Employee.java + Driver.java)

Code Screenshot:

Employee.java

```java
public abstract class Employee {
    protected String name, id;
    public Employee(String name, String id) { this.name = name; this.id = id; }
    public abstract double monthlySalary();
    public String getDetails() { return id + " - " + name; }
}

class FullTime extends Employee {
    private double fixedSalary;
    public FullTime(String n, String id, double s) { super(n, id); this.fixedSalary = s; }
    public double monthlySalary() { return fixedSalary; }
}

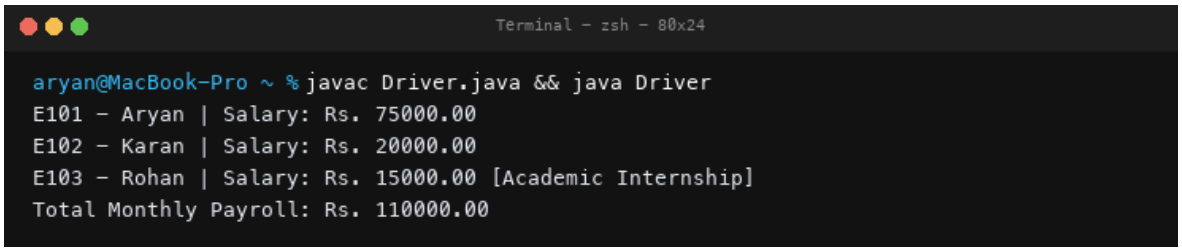
class PartTime extends Employee {
    private double hours, rate;
    public PartTime(String n, String id, double h, double r) { super(n, id); this.hours = h; this.rate = r; }
    public double monthlySalary() { return hours * rate; }
}

class Intern extends Employee {
    private double stipend;
    public Intern(String n, String id, double s) { super(n, id); this.stipend = s; }
    public double monthlySalary() { return stipend; }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Employee[] staff = {
            new FullTime("Aryan", "E101", 75000),
            new PartTime("Karan", "E102", 40, 500),
            new Intern("Rohan", "E103", 15000)
        };
        double totalPayroll = 0;
        for (Employee e : staff) {
            double sal = e.monthlySalary();
            totalPayroll += sal;
            String note = (e instanceof Intern) ? " [Academic Internship]" : "";
            System.out.printf("%s | Salary: Rs. %.2f%s\n", e.getDetails(), sal, note);
        }
        System.out.printf("Total Monthly Payroll: Rs. %.2f\n", totalPayroll);
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned constructor delegation to superclasses using super(...).
- Understood method overriding and specialized behavior in subclasses.
- Learned how and when to use the instanceof type comparison operator.
- Implemented polymorphic payroll computation.
- Practiced clean code structuring with abstract base classes.